CO1-AT1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform

Solutions

Krisha Meenatchi M (192511338)

Question (a): Input Parameters, Basic Operations & Execution Locations

The table below details the input parameter, basic operation, code location, and input dependency for Modules M1 and M2:

Module	Input Size (n)	Basic Operation	Code Location	Variation for Same n
M1: Duplicate Detector	n (items in P)	String comparison P[i].SKU == P[j].SKU	Inner loop line 4	No — always executes n(n-1)/2 comparisons
M2: API Rate Limiter	n (rate buckets)	Logging call LogAccess()	Lines 3 and 6	No — recurrence depends strictly on integer n

Question (b): Exact Mathematical Derivations for M1 and M2

1. Derivation for Module M1 (Duplicate Detector)

The outer loop index i ranges from 0 to n-2. For each i, the inner loop index j ranges from i+1 to n-1, executing exactly 1 comparison per iteration:

$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$
$C(n) = \sum_{i=0}^{n-2} ((n - 1) - (i + 1) + 1) = \sum_{i=0}^{n-2} (n - 1 - i)$

Expanding the sum:
$C(n) = (n - 1) + (n - 2) + ... + 2 + 1 = n(n - 1) / 2 = 0.5\ n^2 - 0.5\ n = \Theta(n^2)$

Exact operation counts:

• At N = 100,000: C(100,000) = (100,000 × 99,999) / 2 = 4,999,950,000 basic operations.
• At N = 1,000,000: C(1,000,000) = (1,000,000 × 999,999) / 2 = 499,999,500,000 basic operations.

2. Derivation for Module M2 (API Rate Limiter Audit)

The recurrence for basic operation LogAccess() counts is:
$A(n) = 2\,A(\lfloor n/2 \rfloor) + 1$ for $n > 1$, with base case $A(1) = 1$.

Case 1: Exact solution for $n = 2^k$:
$A(2^k) = 2\,A(2^{k-1}) + 1 = 2(2\,A(2^{k-2}) + 1) + 1 = 2^2\,A(2^{k-2}) + 2 + 1$
Expanding after k steps:
$A(2^k) = 2^k\,A(1) + \sum_{j=0}^{k-1} 2^j = 2^k + (2^k - 1) = 2^{k+1} - 1 = 2n - 1 = \Theta(n)$

Case 2: Closed-form for arbitrary positive integer n:
$A(n) = 2^{\lfloor \log_2(n-1) \rfloor + 1} - 1$

Numerical Verification for n = 1,024 (2^{10}):
• Using formula $A(n) = 2n - 1$: $A(1,024) = 2(1024) - 1 = 2,047$ ops.
• Note: If audit count counts recursive sub-calls excluding initial call, $A_{audit}(1024) = 1023$ (Formula $n - 1 = 1023$). Verified exact match!

Critique of Colleague's Claim:
The colleague claimed M2 is $\Theta(n \log n)$ by assuming recursion depth $\log_2(n)$ multiplies n work per level. This is mathematically incorrect. At recursion level i, there are 2^i subproblems each doing O(1) work. The total work is the sum of geometric series $\sum_{i=0}^{k} 2^i = 2^{k+1} - 1 = \Theta(n)$. Therefore, M2 is strictly linear $\Theta(n)$, and the colleague's claim is rejected.

Question (c): Binary Search Recurrence & Memory Overhead Analysis

1. Recurrence Relation & Backward Substitution:
$T(n) = T(\lfloor n/2 \rfloor) + 1$ for $n > 1$, with $T(1) = 1$.

Applying backward substitution for $n = 2^k$:
$T(n) = T(n/2) + 1$
$T(n) = [T(n/4) + 1] + 1 = T(n/4) + 2$
$T(n) = T(n/2^i) + i$
Setting $n/2^k = 1 \Longrightarrow k = \log_2(n)$:
$T(n) = T(1) + \log_2(n) = 1 + \log_2(n) = \lfloor \log_2 n \rfloor + 1 = \Theta(\log n)$

2. Auxiliary Memory Space Comparison at N = 1,000,000:
• Iterative Binary Search: Maintains 3 index pointers (low, high, mid). Memory usage is O(1)

auxiliary space (24 bytes total).
• Recursive Binary Search: Creates 1 stack frame per call. Maximum stack depth is $\lfloor \log_2(1{,}000{,}000) \rfloor + 1 = 20$ frames. At ~64 bytes/frame, stack consumption is ~1.28 KB.
• Conclusion: While 1.28 KB is small, iterative O(1) is strictly preferable for high-throughput, memory-constrained microservices to eliminate stack overflow risks.

Question (d): Master Theorem & Recursion Tree Analysis for M4 and M5

1. Module M4 Recommendation Engine (Current): $T(n) = 4\,T(n/2) + n$
• Master Theorem Form: $T(n) = a\,T(n/b) + f(n)$ with $a = 4$, $b = 2$, $f(n) = n$.
• Critical Exponent: $n^{\log_b a} = n^{\log_2 4} = n^2$.
• Comparison: $f(n) = n = O(n^{2 - \varepsilon})$ for $\varepsilon = 1 > 0$.
• Result: Case 1 applies $\Longrightarrow T(n) = \Theta(n^2)$.

2. Module M4 Recommendation Engine (Proposed): $T(n) = 2\,T(n/2) + n$
• Parameters: $a = 2$, $b = 2$, $f(n) = n$.
• Critical Exponent: $n^{\log_b a} = n^{\log_2 2} = n^1 = n$.
• Comparison: $f(n) = \Theta(n^{\log_b a} \log^0 n)$.
• Result: Case 2 applies with $k = 0 \Longrightarrow T(n) = \Theta(n \log n)$.
• Performance Gain at N = 1,000,000: Execution time drops from 20,000 seconds (5.56 hours) to 0.209 seconds. Speedup factor: ~95,550×!

3. Module M5 Analytics Engine: $T(n) = 2\,T(n/2) + n \log_2(n)$
• Master Theorem Failure: Here $a = 2$, $b = 2 \Longrightarrow n^{\log_2 2} = n$. The ratio $f(n)/n = \log_2(n)$. Since $\log_2(n)$ grows faster than 1 but slower than n^{ε} for any $\varepsilon > 0$, standard Master Theorem cases do NOT apply.
• Recursion Tree Derivation:
  - Level 0 (Root): 1 problem of size $n \Longrightarrow$ Work $= n \log_2(n)$
  - Level 1: 2 subproblems of size $n/2 \Longrightarrow$ Work $= 2 \times (n/2 \log_2(n/2)) = n(\log_2 n - 1)$
  - Level i: 2^i subproblems of size $n/2^i \Longrightarrow$ Work $= 2^i \times (n/2^i \log_2(n/2^i)) = n(\log_2 n - i)$
  - Total Tree Height: $H = \log_2(n)$
  - Summing across all levels $i = 0$ to $\log_2(n) - 1$:
    $T(n) = \sum_{i=0}^{\log_2(n)-1} n(\log_2 n - i) + \Theta(n) = n \sum_{j=1}^{\log_2 n} j + \Theta(n)$
    $T(n) = n [\log_2(n)(\log_2(n) + 1) / 2] + \Theta(n) = 0.5\, n \log_2^2(n) + 0.5\, n \log_2(n) + \Theta(n)$
  - Final Asymptotic Bound: $T(n) = \Theta(n \log^2 n)$.

Question (e): Best, Worst, and Average Case Analysis for Sequential Search S1

1. Case Definitions:
• Best Case: Target is at index 0 $\Longrightarrow$ $C_{best}(n) = 1 = O(1)$.
• Worst Case: Target is at index n-1 or not present $\Longrightarrow$ $C_{worst}(n) = n = O(n)$.

2. Average Case Derivation with Presence Probability p:
• If target is present (probability p), it is equally likely at any position 1 to n. Average comparisons = $(n + 1) / 2$.
• If target is absent (probability 1 - p), search scans all n items. Comparisons = n.
• Expected Comparisons Formula:
$C_{avg}(n) = p \times [(n + 1) / 2] + (1 - p) \times n$
$C_{avg}(n) = (p\,n + p + 2\,n - 2\,p\,n) / 2 = n - [p(n - 1) / 2]$

3. Evaluation at N = 1,000,000 and p = 0.5:
$C_{avg}(1{,}000{,}000) = 1{,}000{,}000 - [0.5 \times (999{,}999) / 2] = 1{,}000{,}000 - 249{,}999.75 = 750{,}000.25$ comparisons.
At 10^8 ops/s, execution time is 0.0075 seconds (7.5 ms).

Question (f): Formal Big-Theta Proof and Limit Test

Function to analyze: $f(n) = 3\,n^2 + 10\,n \log_2(n) + 50$

1. Proof that $f(n) = \Theta(n^2)$:
Goal: Find positive constants c1, c2, n0 such that $c1\,n^2 \leq 3\,n^2 + 10\,n \log_2(n) + 50 \leq c2\,n^2$ for all $n \geq n0$.

• Lower Bound (c1):
For $n \geq 1$, $\log_2(n) \geq 0 \Longrightarrow 10\,n \log_2(n) \geq 0$ and $50 > 0$.
Therefore, $3\,n^2 + 10\,n \log_2(n) + 50 \geq 3\,n^2$ for all $n \geq 1$.
Hence, $c1 = 3$ with $n0 = 1$.

• Upper Bound (c2):
For $n \geq 8$, $\log_2(n) \leq n \Longrightarrow 10\,n \log_2(n) \leq 10\,n^2$.
Also for $n \geq 8$, $50 \leq n^2$ (since $8^2 = 64 > 50$).
Therefore, $f(n) \leq 3\,n^2 + 10\,n^2 + n^2 = 14\,n^2$ for all $n \geq 8$.
Hence, $c2 = 14$ with $n0 = 8$.

• Conclusion: Choosing $c1 = 3$, $c2 = 14$, and $n0 = \max(1, 8) = 8$, we have:
$3\,n^2 \leq 3\,n^2 + 10\,n \log_2(n) + 50 \leq 14\,n^2$ for all $n \geq 8$.

Thus, f(n) = Θ(n^2). Q.E.D.

2. Limit Test Proof showing f(n) ≠ O(n):
Lim_{n → ∞} [f(n) / n] = Lim_{n → ∞} [(3 n^2 + 10 n log_2(n) + 50) / n]
Lim_{n → ∞} [3 n + 10 log_2(n) + (50 / n)] = ∞ + ∞ + 0 = ∞.
By definition, if f(n) = O(n), the limit must be a finite constant L < ∞. Since the limit is ∞, f(n) is NOT O(n). Q.E.D.

Question (g): Master Scalability Audit & SLA Evaluation

The table below summarizes the basic operation counts, execution times at 10^8 ops/s, and SLA compliance (600s window) for all platform modules:

Module Name	Asymptotic Bound	N = 100,000 Ops	Time (s)	N = 1,000,000 Ops	SLA Status (600s)
M1: Duplicate Detector	$\Theta(n^2)$	5.000×10^9	50.00 s	5.000×10^{11}	**FAIL (83.3 min)**
M2: API Rate Limiter	$\Theta(n)$	1.311×10^5	0.001 s	1.049×10^6	**PASS (10.5 ms)**
S1: Sequential Search (p=0.5)	$\Theta(n)$	7.500×10^4	0.001 s	7.500×10^5	**PASS (7.5 ms)**
S2: Binary Search	$\Theta(\log n)$	17 ops	0.000 s	20 ops	**PASS (0.2 µs)**
M4: Rec. Engine (Current)	$\Theta(n^2)$	2.000×10^{10}	200.00 s	2.000×10^{12}	**FAIL (5.56 hrs)**
M4: Rec. Engine (Proposed)	$\Theta(n \log n)$	1.761×10^6	0.018 s	2.093×10^7	**PASS (0.209 s)**
M5: Analytics Engine	$\Theta(n \log^2 n)$	1.472×10^7	0.147 s	2.096×10^8	**PASS (2.096 s)**
M5: Report Generator	$\Theta(n^2)$	3.002×10^{10}	300.17 s	3.000×10^{12}	**FAIL (8.33 hrs)**